

SELF-REFINE GUARDRAIL: A Two-Gate Policy, Confidentiality & Error Verification Layer for Generative AI Customer Support Chatbots

VIDHURSH KUMAR V, Final Year Student, Department of Artificial Intelligence & Data Science, Rajalakshmi Engineering College, Thandalam, Sriperumbudur, Tamil Nadu, India.

MEENAKSHI SS, Final Year Student, Department of Artificial Intelligence & Data Science, Rajalakshmi Engineering College, Thandalam, Sriperumbudur, Tamil Nadu, India.

JASHAREEN J, Final Year Student, Department of Artificial Intelligence & Data Science, Rajalakshmi Engineering College, Thandalam, Sriperumbudur, Tamil Nadu, India.

E. Rajalakshmi, Assistant Professor, Department of Artificial Intelligence & Data Science, Rajalakshmi Engineering College, Thandalam, Sriperumbudur, Tamil Nadu, India.

Abstract - Generative artificial intelligence is increasingly used in customer-support systems, where fluent generation alone does not guarantee a safe response. A candidate answer may contain factual errors, violate organizational policy, expose recognizable sensitive information, be incomplete, or adopt an inappropriate tone. This paper presents Self-Refine Guardrail (SRG), a response-level verification layer positioned between a generative support model and the customer. The current architecture uses two structurally isolated verification gates. Gate 1, the Review Agent, receives a redacted candidate response and relevant policy context and evaluates policy compliance, tone, and completeness. Gate 2, the Error Checker, receives the original response and evaluates factual correctness and confidentiality. Confidentiality verification begins with deterministic regular-expression checks for recognizable sensitive-data patterns and can use semantic evaluation for contextual reasoning. The two Boolean gate verdicts are combined by deterministic application logic using $\text{approve} = v1 \text{ AND } v2$. A response is delivered only when both gates pass. Otherwise, the failing gate's reason is used by a bounded Self-Refine process to generate a revised response, which is fully reverified. A maximum of two refinement attempts is permitted; persistent failure results in human escalation. The current implementation uses Python, FastAPI, a Groq-based LLM client with openai/gpt-oss-20b, sentence-transformers, FAISS, n8n, and a lightweight HTML/CSS/JavaScript frontend. Preliminary evaluation on 20 synthetic cases produced TP=4, TN=16, FP=0, and FN=0, yielding 100% precision, recall, and F1-score. All 16 initially denied cases were successfully refined, with zero escalations. These findings represent preliminary validation of the implemented pipeline and do not establish production-level reliability or generalization.

Keywords—generative AI, customer-support chatbot, LLM guardrails, confidentiality, policy compliance, factual verification, Self-Refine, retrieval-augmented generation, FastAPI, n8n.

I. INTRODUCTION

Generative AI has introduced a flexible paradigm for customer-support automation. Unlike conventional rule-based systems, a support LLM can interpret natural-language questions, synthesize information, and produce responses that adapt to conversational context. This flexibility is valuable for customer service, but it also changes the reliability problem. A generated response is not guaranteed to be correct merely because it is fluent, relevant, or confidently expressed.

In customer-support environments, an erroneous response can have consequences beyond user dissatisfaction. A chatbot may state an organizational policy incorrectly, expose a customer's sensitive information, omit an important condition, or comply with a user instruction that conflicts with the intended behavior of the service. The problem is therefore not only how to generate a response, but also how to verify the generated response before it becomes externally visible.

Real-world incidents demonstrate this distinction. In *Moffatt v. Air Canada*, a chatbot provided incorrect information regarding a bereavement-fare policy. The incident illustrates how an apparently useful conversational answer can conflict with the organization's actual policy. In another widely discussed incident involving a Chevrolet dealership chatbot, prompting was used to induce the chatbot to agree to an inappropriate vehicle-sale scenario. These incidents differ in domain, but both illustrate the need for controls around generated outputs rather than relying exclusively on the generator.

Recent research addresses this problem through hallucination detection, LLM guardrails, retrieval-augmented generation, LLM-based evaluation, and iterative refinement. Hallucination surveys identify factual inconsistency as a persistent challenge for large language models [1]. Guardrail research argues for systematic mechanisms that constrain or evaluate model behavior [2], [3]. Retrieval-augmented generation provides access to external knowledge that can ground model reasoning [4]. LLM-as-a-judge research studies the use of language models as evaluators while also identifying reliability and consistency concerns [5], [6]. Error-detection research demonstrates that evaluating generated responses is itself a difficult task [7]. Self-Refine provides a framework for iterative feedback and rewriting [8].

SRG combines these directions around a response-level verification boundary. The current design deliberately avoids assigning every concern to a single evaluator. The Review Agent evaluates policy compliance, tone, and completeness using a redacted response. The Error Checker evaluates factual correctness and confidentiality using the original response. This distinction allows each gate to operate on the information needed for its assigned responsibility.

The final decision is intentionally deterministic. If v_1 is the Boolean verdict from the Review Agent and v_2 is the Boolean verdict from the Error Checker, the application computes $\text{approve} = v_1 \text{ AND } v_2$. The LLM does not directly authorize delivery. If either gate fails, the candidate is denied and enters Self-Refine. The refined candidate is reverified through the complete pipeline, and autonomous correction is bounded to two attempts.

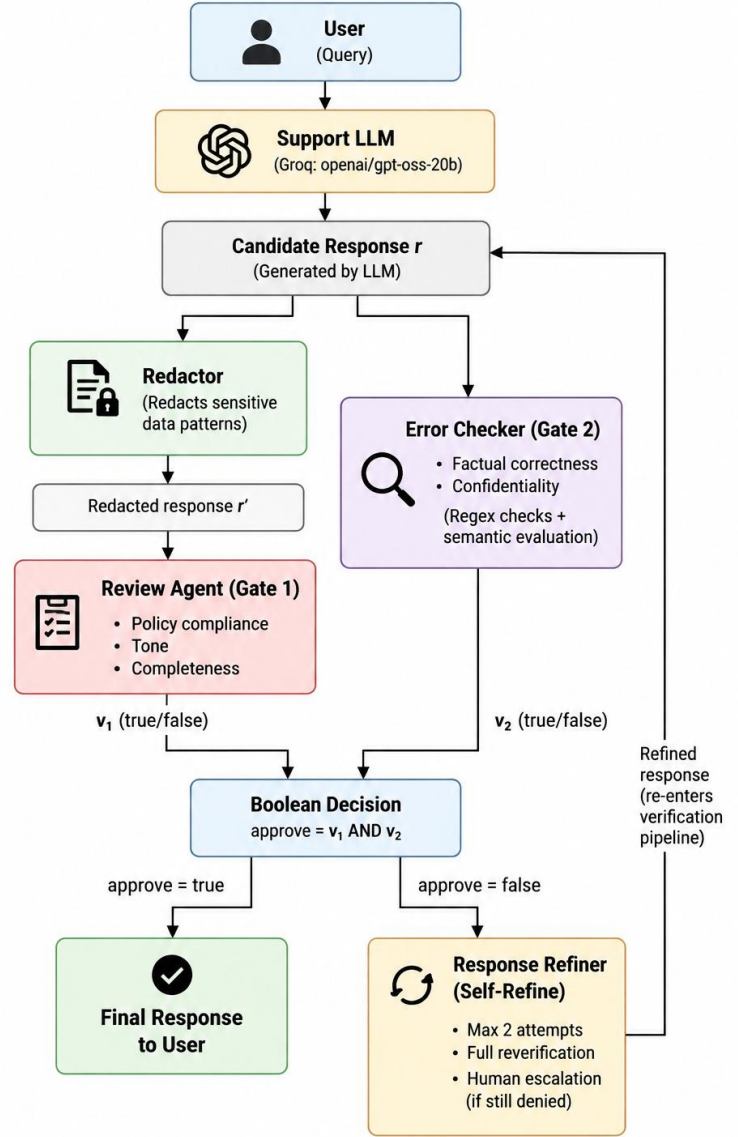


Fig. 1. Overall SRG Two-Gate Architecture

A. Contributions

The main contribution is a practical architecture that separates policy-oriented review from factual and confidentiality verification while preserving a single deterministic delivery condition. The design also contributes a Redactor stage for data minimization, a bounded Self-Refine mechanism with full reverification, and a structured integration contract suitable for a customer-support application. The paper documents the implemented system and reports a small synthetic evaluation without extending the results beyond what the evaluation supports.

II. RELATED WORK

A. LLM Guardrails and Safeguarding

Guardrails have emerged as an important layer for controlling the behavior of generative systems. Dong et al. describe systematic guardrail design as a way to address application-specific risks around LLM inputs and outputs [2]. Their work emphasizes that guardrails should be designed according to the risk characteristics of the application rather than treated as a generic filter. A broader survey by Dong et al. reviews safeguarding techniques covering privacy, attacks, hallucination, and defense mechanisms [3].

SRG follows the response-level perspective of these studies. Instead of assuming that the support model will always obey policy, it inserts an explicit verification stage after generation. The current architecture further divides the verification task into two gates, making the boundary between policy/tone/completeness and factual/confidentiality checking explicit.

B. Retrieval-Augmented Policy Grounding

Lewis et al. introduced retrieval-augmented generation as a method that combines parametric language models with external non-parametric knowledge sources [4]. Retrieval is particularly relevant to customer-support systems because organizational policies may be detailed, domain-specific, and subject to change. In SRG, policy retrieval supplies relevant context to the Review Agent so that policy compliance is judged against retrieved information rather than solely against model memory.

The current implementation uses sentence-transformer embeddings and FAISS vector search over policy documents and chunks. The retriever is an implemented component of the system. However, retrieval performance is not separately quantified in the present evaluation, and no retrieval accuracy or latency claim is made.

C. Error Detection in LLM Responses

Kamoi et al. investigate the ability of LLMs to detect errors in generated responses and show that response evaluation remains challenging [7]. This observation is directly relevant to SRG because the Error Checker is itself model-assisted for semantic reasoning. The architecture therefore does not treat semantic evaluation as infallible. Deterministic

checks are retained for recognizable sensitive patterns, and the final routing rule remains outside the evaluator model.

D. LLM-as-a-Judge

Zheng et al. evaluate LLM-as-a-judge behavior using MT-Bench and Chatbot Arena [5]. Gu et al. survey the broader LLM-as-a-judge paradigm and discuss reliability, consistency, bias, and standardization challenges [6]. SRG uses the evaluator paradigm in a constrained form: each gate has a narrow responsibility, and the evaluator produces a Boolean verdict plus a concise reason. The reason is feedback and trace information, not the final authorization signal.

E. Self-Refine

Madaan et al. introduce Self-Refine, in which a model generates an output, produces feedback, and iteratively improves the output [8]. SRG adopts the iterative principle but places it inside a safety boundary. Refinement occurs only after a failed verification, uses the reason associated with the failing concern, and is followed by complete reverification. The refined response therefore cannot bypass the guardrail.

F. Hallucination Detection

Huang et al. survey hallucination in large language models and characterize the broad problem of factual inconsistency and unreliable generation [1]. Manakul et al. propose SelfCheckGPT as a black-box approach for hallucination detection based on consistency across sampled responses [9]. SRG is complementary to these approaches: its Error Checker evaluates the factual correctness of the candidate in the context of the customer-support workflow, while the overall architecture adds policy, confidentiality, and refinement controls.

G. Research Gap

The literature provides substantial work on guardrails, retrieval, response evaluation, hallucination detection, and iterative refinement. The project focuses on their architectural composition for customer-support response verification. The specific design emphasizes two isolated evaluation gates, asymmetric information visibility through the Redactor, deterministic Boolean authorization, and

bounded Self-Refine with reverification. This provides a concrete framework in which each concern can be evaluated independently while preserving a simple final acceptance condition.

Ref.	Approach / Work	Primary Focus	Relation to SRG
[1]	LLM hallucination survey	Hallucination and reliability	Motivates factual response verification
[2]	LLM guardrails	Systematic safeguarding	Motivates response-level guardrail layer
[3]	LLM safeguarding survey	Privacy, attacks, hallucination	Supports layered safety design
[4]	RAG	External knowledge retrieval	Provides policy-context retrieval basis
[5]	LLM-as-a-judge	LLM response evaluation	Motivates constrained semantic evaluation
[6]	LLM-as-a-judge survey	Reliability and consistency	Motivates evaluator isolation
[7]	LLM error detection	Detecting response errors	Supports Error Checker role
[8]	Self-Refine	Iterative feedback	Provides refinement basis
[9]	SelfCheckGPT	Hallucination detection	Related post-generation verification

Table I. Comparison of Related Work with the Proposed SRG.

III. PROBLEM FORMULATION AND DESIGN OBJECTIVES

Let q denote a user request and r denote the candidate response generated by the support LLM. The objective of SRG is to determine whether r can be delivered safely after independent verification of policy-oriented and error/confidentiality-oriented properties. The system is not designed to replace the support LLM; rather, it controls the boundary at which the generated response becomes deliverable.

The first verification function produces $v1$ from the redacted response r' and relevant policy context P . The Review Agent considers policy compliance, tone, and completeness. The second verification function produces $v2$ from the original response r and considers factual correctness and confidentiality. The final approval function is:

$$\text{approve} = v1 \wedge v2$$

The conjunction means that approval is possible only when both gate verdicts are true. No numerical risk score, weighted aggregation, or threshold is used in the current architecture. The decision table is therefore small, deterministic, and directly testable.

$v1$	$v2$	Decision	Reason
true	true	APPROVED	Both independent gates pass.
true	false	DENIED	Gate 2 fails factual/confidentiality verification.
false	true	DENIED	Gate 1 fails policy/tone/completeness review.
false	false	DENIED	Both gates fail.

Table II. Deterministic Two-Gate Decision Matrix.

A. Design Objectives

- Verify every candidate response before delivery.
- Separate policy/tone/completeness review from factual/confidentiality checking.
- Reduce unnecessary exposure of recognizable sensitive spans through deterministic redaction.
- Retain the original response for confidentiality verification.
- Keep final routing outside the LLM using a Boolean conjunction.
- Allow correction through Self-Refine without bypassing reverification.
- Limit autonomous refinement to two attempts.
- Escalate persistent failures instead of silently delivering them.

B. Functional Requirements

ID	Requirement	Description
FR-1	Candidate generation	A support LLM generates the candidate response.
FR-2	Redaction	Recognizable sensitive patterns are redacted before Gate 1.
FR-3	Gate 1	Review Agent checks policy, tone, and completeness.
FR-4	Gate 2	Error Checker checks factual correctness and confidentiality.
FR-5	Boolean authorization	Approval is computed as $v1 \text{ AND } v2$.
FR-6	Reverification	Every refined candidate passes the full pipeline again.
FR-7	Attempt bound	At most two refinement attempts are allowed.
FR-8	Human fallback	Persistent denial enters human review.

Table III. Core Functional Requirements.

IV. PROPOSED SYSTEM ARCHITECTURE

A. Overall Architecture

SRG is positioned between the support LLM and the customer-facing response channel. The support model produces a candidate response, but that response is treated as untrusted until it passes two independent verification gates. The architecture separates preprocessing, semantic evaluation, deterministic routing, and iterative correction.

The candidate r enters the Redactor. The Redactor creates r' by replacing recognizable sensitive spans. The redacted response is sent to the Review Agent together with relevant policy context. In parallel, the original r is provided to the Error Checker. The resulting Boolean verdicts are combined by application logic.

B. Redactor

The Redactor is deterministic and operates before Gate 1. It scans for recognizable sensitive-data patterns such as account-number-like strings, email addresses, phone numbers, credit-card-like numbers, and SSN-like numbers. Matching spans are replaced before the Review Agent receives the response.

The Redactor has a limited but important purpose. It minimizes unnecessary exposure of sensitive spans to a reviewer whose task is policy, tone, and completeness. It does not determine the final confidentiality status. The original candidate therefore remains available to the Error Checker, where confidentiality is explicitly evaluated.

C. Review Agent — Gate 1

The Review Agent receives r' and policy context P . Its evaluation scope is limited to policy compliance, tone, and completeness. This scope is structural: the agent is not responsible for deciding whether sensitive information is present in the original response.

The expected structured result contains a Boolean passed field and a concise reason, conceptually represented as $\{\text{"passed": true/false, "reason": "one sentence"}\}$. The application uses the Boolean value as $v1$. The free-form reason is retained for explanation and refinement feedback but cannot override the Boolean result.

D. Error Checker — Gate 2

The Error Checker receives the original response r . It evaluates factual correctness and confidentiality. Confidentiality checking begins with deterministic regular-expression checks. When a recognizable hard sensitive pattern is detected, $v2$ is set to false for that condition and an additional semantic call is not required to establish the hard-pattern failure.

When deterministic checks do not settle the confidentiality question, semantic reasoning can be used for contextual evaluation. The same gate also evaluates factual correctness. Keeping these tasks together in Gate 2 preserves the distinction between error/confidentiality checking and the policy/tone/completeness responsibilities of Gate 1.

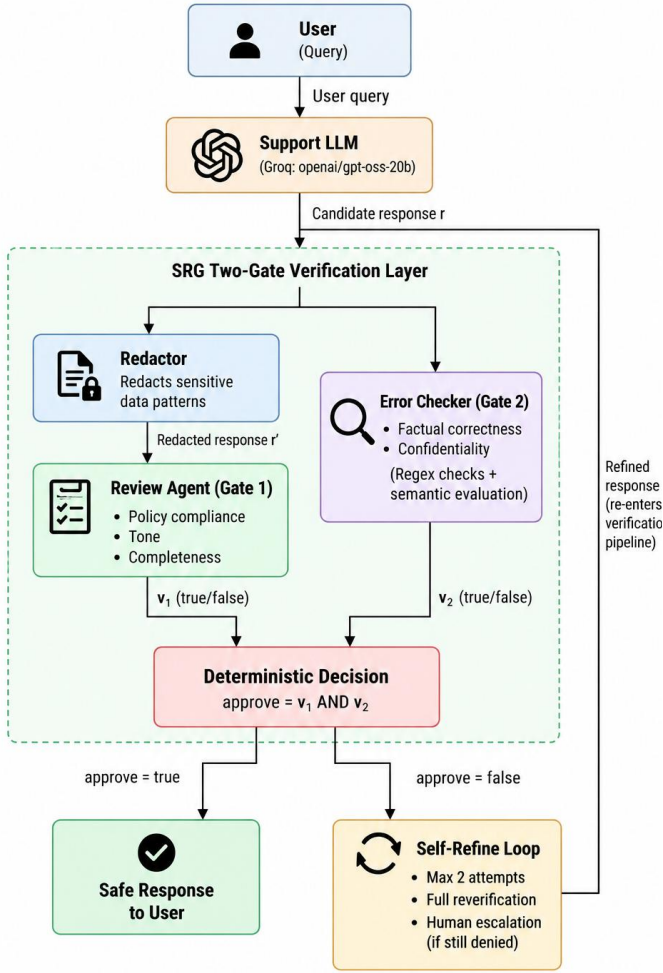


Fig. 2. Two-Gate Information Isolation.

E. Deterministic Boolean Decision

After the two gates complete, the application computes $\text{approve} = v_1 \text{ AND } v_2$. This rule is the sole authorization condition. The LLM does not infer the final routing action from the natural-language reason. A false result from either gate is sufficient to prevent delivery.

F. Two-Gate Isolation

The two-agent design is a structural separation rather than two copies of the same prompt. Gate 1 sees a redacted response and policy context; Gate 2 sees the original response. Their responsibilities are also separated. This reduces cross-concern reasoning leakage, particularly when evaluator explanations are reused as Self-Refine feedback.

A combined evaluator could produce a correct Boolean label while giving an explanation that references evidence belonging to a different concern.

If such an explanation is passed to a refiner, unrelated portions of the answer may be changed. SRG reduces this risk by keeping the evidence and responsibility boundaries explicit.

G. Self-Refine Loop

When approval is false, the response is denied rather than delivered. The reason associated with the failing gate is used to form a rewrite instruction. The Response Refiner produces a revised candidate, which is then processed again through Redactor, Review Agent, Error Checker, and the Boolean decision.

The current system permits a maximum of two refinement attempts. If the response remains denied after the allowed attempts, the system escalates to human review. There is no silent delivery path from a persistent denied state.

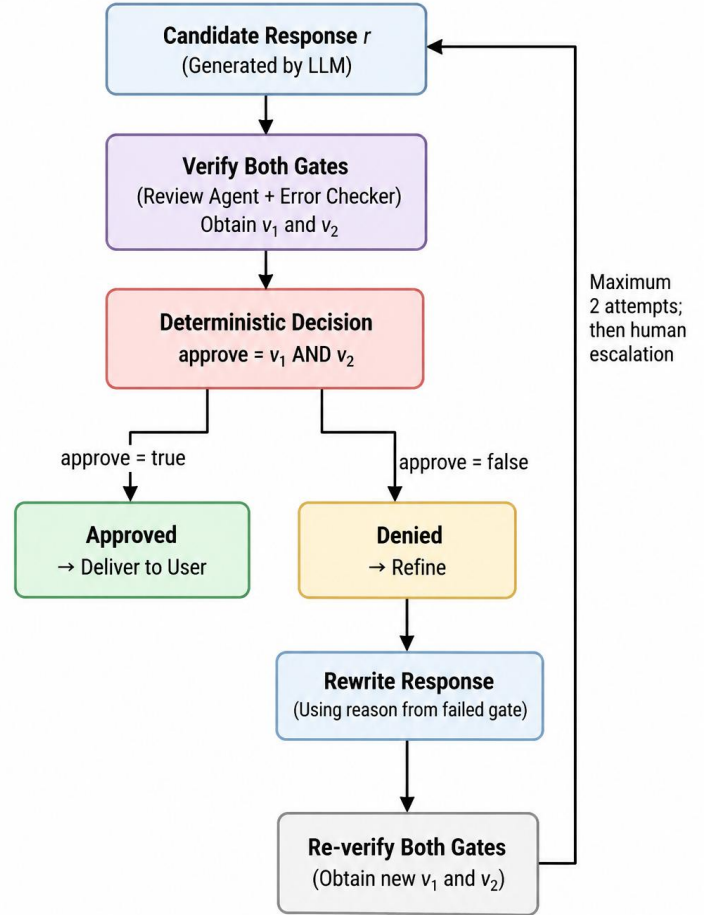


Fig. 3. Self-Refine Reverification Loop.

H. Component Responsibilities

Component	Input	Primary Responsibility	Output
Redactor	Candidate r	Redact recognizable sensitive spans	Redacted r'
Review Agent	r' + policy P	Policy, tone, completeness	v1 + reason
Error Checker	Original r	Factual correctness, confidentiality	v2 + reason
Boolean logic	v1, v2	Deterministic authorization	approve
Response Refiner	Denied r + reason	Generate corrected candidate	Revised r
Policy Retriever	Query/response context	Retrieve relevant policy chunks	Policy context

Table IV. SRG Components and Responsibilities.

V. IMPLEMENTATION AND INTEGRATION

A. Backend Organization

The current backend is implemented in Python using FastAPI. The code organization separates transport routes, schemas, SRG verification modules, retrieval, storage, and testing. The structure includes `main.py` as the application entry point, `routes/chat.py` for chat routing, `schemas/chat_schema.py` for structured data definitions, and the `srg` package for the Redactor, Review Agent, Error Checker, Response Refiner, policy retriever, utilities, and package initialization.

Module	Current Role
<code>main.py</code>	FastAPI application entry point
<code>routes/chat.py</code>	Chat request routing and pipeline coordination
<code>schemas/chat_schema.py</code>	Structured request/response schemas
<code>srg/redactor.py</code>	Sensitive-pattern redaction
<code>srg/review_agent.py</code>	Gate 1 evaluation
<code>srg/error_checker.py</code>	Gate 2 evaluation
<code>srg/response_refiner.py</code>	Bounded response refinement
<code>srg/policy_retriever.py</code>	Policy retrieval using embeddings and FAISS
<code>srg/utils.py</code>	Supporting utilities
<code>storage/faiss_index/</code>	FAISS vector index storage
<code>storage/policy_docs/</code>	Policy document storage
<code>test_srg_core.py</code>	Core SRG tests

Table V. Current Backend Module Organization

B. LLM Integration

The current LLM integration uses a Groq-based LLM client with the model `openai/gpt-oss-20b`. The model is used where generation or semantic reasoning is required by the implemented pipeline. The architecture does not treat model output as an authorization signal. Instead, model-assisted gate judgments are converted into structured Boolean values and passed to deterministic application logic.

This design also avoids making the guardrail core dependent on one particular evaluator implementation. The important contract is that the relevant component produces the required structured result. The current project, however, specifically documents the Groq-based integration and does not describe older mock-only or alternate-model configurations as the current implementation.

C. Policy Retrieval

The policy retrieval component uses sentence-transformer embeddings and FAISS vector search. Policy documents are represented as retrievable chunks and indexed in a vector store. During a review, relevant policy context can be supplied to the Review Agent. The retriever provides context; it does not itself make the policy verdict.

The retrieval component should be distinguished from retrieval evaluation. It is implemented in the current project, but no retrieval precision, recall, coverage, or latency metric is claimed because those measures were not part of the authoritative 20-case evaluation.

D. FastAPI Service

FastAPI provides the service boundary used by the chatbot integration. The chat route coordinates candidate processing and returns structured information needed by downstream components. The response contract includes fields such as `final_response`, `approved`, `v1`, `v2`, `reason`, `attempts_used`, and `escalated`.

E. n8n Workflow

The current workflow orchestration layer is n8n and is operated locally/self-hosted during development. The workflow is named SRG Guardrail Pipeline. Its current node sequence is Webhook → HTTP Request → Respond to Webhook. The Webhook accepts POST requests at

the path `srg-chat`. The HTTP Request node forwards `user_input` to the FastAPI `/chat` endpoint, and Respond to Webhook returns the resulting structured response.

A successful production-path test returned structured fields including `final_response`, `approved`, `v1`, `v2`, `reason`, `attempts_used`, and `escalated`. This demonstrates the integration path without moving the guardrail decision logic into `n8n`.

F. Frontend

The frontend is implemented using HTML, CSS, and JavaScript. It accepts user messages, sends them to the `n8n` webhook, displays the final response, and presents SRG verification status. The interface includes concepts corresponding to overall state, confidentiality, policy/review, error checking, and Self-Refine. The frontend does not implement the Boolean authorization logic.

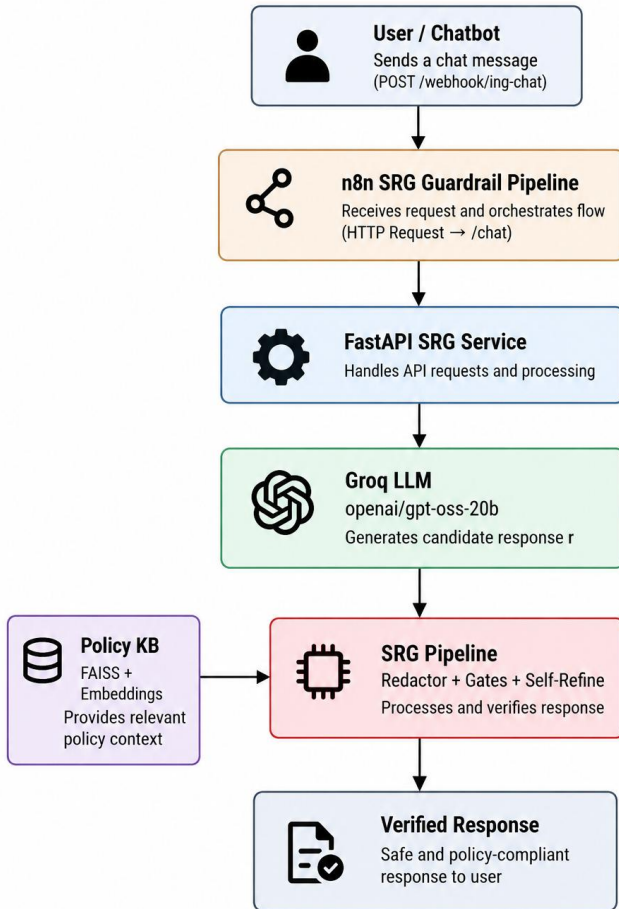


Fig. 4. Current Deployment and Integration Architecture.

G. Current Implementation Summary

Layer	Technology / Component	Current Status
Backend	Python + FastAPI	Implemented
LLM	Groq + openai/gpt-oss-20b	Current integration
Policy retrieval	sentence-transformers + FAISS	Implemented component
Verification	Redactor + Review Agent + Error Checker	Implemented architecture
Refinement	Response Refiner, max. two attempts	Implemented
Workflow	n8n, SRG Guardrail Pipeline	Current development integration
Frontend	HTML, CSS, JavaScript	Implemented interface
Deployment	Local/self-hosted development path	Current development configuration

Table VI. Current Implementation Summary.

VI. END-TO-END OPERATION

The SRG pipeline begins when a user submits a support request. The support LLM generates a candidate response `r`. At this point, `r` is not trusted for delivery. The candidate is passed to the Redactor, which creates `r'` for Gate 1 while preserving `r` for Gate 2.

The Review Agent receives `r'` and relevant policy context. It evaluates policy compliance, tone, and completeness and returns `v1` with a concise reason. In parallel, the Error Checker receives `r` and evaluates factual correctness and confidentiality. Deterministic sensitive-pattern checks are performed first for recognizable hard patterns; semantic evaluation is used when contextual reasoning is needed.

The application then evaluates `approve = v1 AND v2`. If both values are true, the response is approved and delivered. If either value is false, the candidate

is denied. The failing gate's reason is used to construct refinement feedback. The Response Refiner creates a revised candidate, which is reintroduced into the same pipeline.

The process is bounded. The system may perform at most two refinement attempts. If a refined response passes both gates, it can be delivered. If it remains denied after the maximum attempts, the system escalates to human review. This sequence preserves the principle that correction does not create an exception to verification.

A. Information Flow

The asymmetric information flow is a central property of the design. Gate 1 receives r' because policy, tone, and completeness do not normally require access to exact sensitive spans. Gate 2 receives r because confidentiality verification must inspect the actual content that would otherwise be delivered. This means the same candidate is evaluated from two controlled views rather than exposing the full response to every component.

B. Structured State

The application maintains a structured state containing the final response and verification information. `approved` records the deterministic final outcome; `v1` and `v2` record the gate outcomes; `reason` provides concise explanatory information; `attempts_used` records the refinement count; and `escalated` indicates whether the bounded process reached human review. These fields provide an explicit integration contract for `n8n` and the frontend.

C. Algorithm

Algorithm 1 describes the logical sequence. The pseudocode represents the architecture rather than any particular prompt implementation.

Input: candidate response r and policy knowledge base K

1. $r' \leftarrow \text{Redactor}(r)$
2. $P \leftarrow \text{RetrievePolicy}(K, r')$
3. $(v1, e1) \leftarrow \text{ReviewAgent}(r', P)$
4. $(v2, e2) \leftarrow \text{ErrorChecker}(r)$
5. $\text{approve} \leftarrow v1 \text{ AND } v2$
6. if $\text{approve} = \text{true}$: deliver r
7. else if $\text{refinement attempts} < 2$: refine r using failing reason
8. else: escalate r for human review
9. Reverify every refined response from Step 1.

Algorithm 1. SRG response verification and bounded Self-Refine procedure.

VII. SECURITY AND THREAT ANALYSIS

The security objective of SRG is to prevent an unsafe or unverified generated response from reaching the customer. The architecture addresses several threats through distinct controls rather than relying on one universal detector.

A. Confidential Information Leakage

Confidentiality is addressed by both deterministic and semantic mechanisms. The Redactor minimizes exposure to Gate 1, while the Error Checker evaluates the original response. Recognizable sensitive formats can be rejected deterministically. Semantic evaluation can identify contextual cases that do not match a simple pattern. The two-stage approach reduces dependence on either mechanism alone.

B. Policy Circumvention

A user may attempt to persuade the support model to ignore organizational policy. Because the generated response is evaluated after generation, a prompt-induced policy violation must still pass the Review Agent. Retrieved policy context provides evidence for this evaluation. The architecture therefore adds a post-generation control point even when the generator has been manipulated.

C. Factual Errors and Hallucination

Factual errors are explicitly assigned to Gate 2. A response can be fluent while containing an incorrect claim. The Error Checker is responsible for this dimension, and failed factual reasoning can become targeted refinement feedback. The project does not claim that model-based checking is infallible; evaluator error remains a limitation.

D. Obfuscated Sensitive Information

Regular expressions are effective only for recognizable patterns. A malicious or accidental leak can be obfuscated, transformed, split across text, or represented indirectly. Semantic confidentiality reasoning can extend coverage, but it cannot guarantee detection of every unseen representation. Future evaluation should therefore include systematic obfuscation tests.

E. Cross-Concern Evaluator Leakage

When an evaluator handles too many concerns, its explanation can mix evidence. Such mixed feedback can contaminate Self-Refine. SRG addresses this threat through structural separation: Gate 1 sees a redacted response and focuses on policy/tone/completeness, while Gate 2 sees the

original response and focuses on factual/confidentiality concerns.

F. Repeated Refinement

Repeated refinement can increase latency and model-call volume and may introduce new errors. The maximum of two attempts is therefore both a resource boundary and a safety boundary. Persistent denial leads to human escalation rather than unlimited autonomous generation.

Threat	Primary Mitigation	Residual Limitation
Confidential data leakage	Redactor + Error Checker	Novel/obfuscated data may evade detection
Policy circumvention	Review Agent + policy context	Semantic evaluator may miss violations
Factual errors	Error Checker	LLM-based factual checking can be imperfect
Prompt-induced unsafe output	Post-generation verification	Adversarial coverage remains limited
Cross-concern leakage	Two isolated gates	Isolation does not remove all evaluator error
Refinement abuse/failure	Two-attempt bound + human review	Escalation workflow needs production validation

Table VII. Threat Categories and Current Mitigations

VIII. EXPERIMENTAL EVALUATION

A. Dataset and Evaluation Setting

The current evaluation contains 20 synthetic cases. The cases are intended to exercise the implemented response-verification pipeline rather than represent a statistically comprehensive customer-support benchmark. The evaluation is a single run and should therefore be interpreted as preliminary pipeline validation.

B. Classification Evaluation

The classification outcomes are represented using true positives, true negatives, false positives, and false negatives. The authoritative results are TP=4, TN=16, FP=0, and FN=0. These counts sum to 20 cases.

	Predicted Positive	Predicted Negative
Actual Positive	TP = 4	FN = 0
Actual Negative	FP = 0	TN = 16

Table VIII. Confusion Matrix for the 20-Case Evaluation.

$$\text{Precision} = \text{TP}/(\text{TP}+\text{FP}) = 4/(4+0) = 100\%.$$

$$\text{Recall} = \text{TP}/(\text{TP}+\text{FN}) = 4/(4+0) = 100\%.$$

$$\text{F1-score} = 2\text{PR}/(\text{P}+\text{R}) = 100\%.$$

$$\text{False Negative Rate} = \text{FN}/(\text{TP}+\text{FN}) = 0\%.$$

The reported metrics are mathematically consistent with the confusion matrix. However, a perfect result on 20 synthetic cases cannot establish production-level accuracy, universal safety, or generalization. The evaluation should be understood as evidence that the implemented pipeline produced the expected classification outcomes on the tested sample.

C. Self-Refine Evaluation

The refinement analysis is kept separate from classification metrics. Of the 20 cases, 16 were initially denied. All 16 initially denied cases were successfully refined and subsequently passed verification. No case reached human escalation.

Measure	Result
Total cases	20
Initially denied	16
Successful refinements	16
Refinement success rate	100%
Escalations	0
Escalation rate	0%

Table IX. Self-Refine Results.

The refinement success rate is 16/16 = 100%. This metric describes the recovery behavior of the Self-Refine loop and must not be combined with precision, recall, or F1-score, which describe classification performance.

D. Case-Level Interpretation

The classification results matched the expected outcomes for all tested cases, while all 16 initially denied cases were successfully refined within the allowed attempts. These results provide preliminary evidence that the current pipeline is feasible as an integrated prototype. The absence of escalations does not imply that escalation is unnecessary; it only indicates that none of the tested cases remained denied after refinement.

E. Preliminary Nature of the Results

The evaluation is limited by its small sample size, synthetic cases, and single evaluation run. Therefore, the results should not be generalized to unseen

policies, adversarial users, obfuscated sensitive information, or production-scale workloads. Future evaluation should include larger and diverse datasets, repeated runs, adversarial testing, and systematic latency and throughput measurements.

IX. DISCUSSION AND LIMITATIONS

The principal design decision in SRG is to perform verification after generation while maintaining clear responsibility boundaries. The generated output is treated as a candidate response rather than an authorized response. Gate 1 reviews policy compliance, tone, and completeness using the redacted response, while Gate 2 evaluates factual correctness and confidentiality using the original response.

The final decision is deterministic: $approve = v_1 \wedge v_2$. A response is delivered only when both gates pass, making the system conservative and avoiding numerical scores or weighted thresholds.

A. Refinement as Controlled Recovery

Self-Refine is triggered only after verification failure. The failing gate's reason guides the rewrite, and the revised response is passed through the complete verification pipeline again. A maximum of two refinement attempts prevents uncontrolled refinement, after which persistent failures are escalated to human review.

B. Operational Trade-Offs

SRG introduces additional processing through policy retrieval, model-assisted verification, and possible refinement. The current project has not systematically measured latency, throughput, or cost; these remain areas for future evaluation.

C. Policy Knowledge Quality

Policy retrieval provides relevant context to the Review Agent but depends on the quality and completeness of the policy corpus. Outdated or incomplete policies may affect verification. The current implementation includes policy retrieval but does not report retrieval performance.

D. Separation Versus Complexity

Two isolated agents require more orchestration than a single evaluator, but provide clearer responsibility boundaries and reduce cross-concern contamination. This makes the verification process easier to inspect, test, and refine.

E. Limitations and Scope

The evaluation is limited to 20 synthetic cases in a single run. Although perfect classification results were obtained, they do not establish production-level reliability or generalization. Regular-expression checks may miss obfuscated or previously unseen sensitive information, while semantic verification depends on the underlying LLM.

The evaluation also lacks systematic benchmarking of retrieval quality, latency, throughput, and cost. Broader adversarial testing and validation of the human escalation workflow are required.

Overall, SRG demonstrates an implemented two-gate verification architecture with preliminary validation, rather than a fully validated production safety system.

XI. CONCLUSION AND FUTURE WORK

This paper presented Self-Refine Guardrail (SRG), a response-level verification layer for generative AI customer-support chatbots. The architecture uses two isolated gates: the Review Agent evaluates policy compliance, tone, and completeness using the redacted response, while the Error Checker evaluates factual correctness and confidentiality using the original response. Their Boolean outputs are combined as $approve = v_1 \wedge v_2$, ensuring that a response is delivered only when both gates pass.

When a response is denied, its verification reason guides Self-Refine, and the revised response is fully re-verified. The system allows a maximum of two refinement attempts, followed by human escalation if the response remains denied.

The implemented system integrates Python, FastAPI, Groq with openai/gpt-oss-20b, sentence-transformers, FAISS, n8n, and a lightweight web interface. In the preliminary evaluation of 20 synthetic cases, the system achieved 100% precision, recall, and F1-score, with 0% false negative rate. All 16 initially denied cases were successfully refined, with zero escalations. These results provide preliminary validation but do not establish production-level reliability.

A. Future Work

Future work will focus on:

- Evaluating SRG on larger and more diverse datasets with repeated runs
- Testing adversarial prompts, policy circumvention, and obfuscated sensitive information.
- Measuring policy retrieval quality, latency, throughput, model-call count, and cost.
- Evaluating alternative LLM evaluators and stronger confidentiality detection.
- Studying refinement-induced errors and validating the human escalation workflow.

XII. REFERENCES

- [1] Y. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A Survey on Hallucination in Large Language Models,” *ACM Transactions on Information Systems*, vol. 43, no. 2, Art. no. 42, pp. 1–55, 2025.
- [2] Y. Dong, R. Mu, G. Jin, Y. Qi, J. Hu, X. Zhao, J. Meng, W. Ruan, and X. Huang, “Position: Building Guardrails for Large Language Models Requires Systematic Design,” *Proc. 41st Int. Conf. on Machine Learning*, vol. 235, pp. 11375–11394, 2024.
- [3] Y. Dong, R. Mu, Y. Zhang, S. Sun, T. Zhang, C. Wu, G. Jin, Y. Qi, J. Hu, J. Meng, S. Bensalem, and X. Huang, “Safeguarding Large Language Models: A Survey,” *Artificial Intelligence Review*, vol. 58, Art. no. 382, 2025.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [5] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [6] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, “A Survey on LLM-as-a-Judge,” *arXiv:2411.15594*, 2024.
- [7] R. Kamoi et al., “Evaluating LLMs at Detecting Errors in LLM Responses,” *Proc. Conference on Language Modeling (COLM)*, 2024.
- [8] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [9] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models,” *Proc. 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9004–9017

